EMP

Employee Management Portal

Viva Preparation & Complete Project Walkthrough

Angular 22 · Standalone · Zoneless · SSR · Nx workspace

98 TypeScript files · ~11,000 lines · 100 tests

Angular Advanced Training — Practical Assessment

0. How to use this document

This is written to be *read out loud*. Section 1 is what to say in the first minute. Section 3 is the single story that explains the whole architecture — if you learn one thing, learn that. Section 4 walks every file. Section 6 is the questions you will actually be asked.

> **Green boxes** are suggested wording — things you can say almost verbatim.

> **Amber boxes** are honest limitations. Volunteering these makes you look stronger, not weaker. An examiner who finds a weakness you already named is impressed; one who finds a weakness you hid is not.

Contents

1. The opening minute

When they say "*tell us about your project*", say roughly this:

> **Say:** "It's an internal HR portal — an Employee Management Portal — built with Angular 22. The main job is managing people: a searchable directory, full add/edit/delete, attendance, leave with approvals, departments, and an admin area.
>
> Architecturally it's standalone components throughout, zoneless change detection, and signal-based state using NgRx SignalStore. It renders on the server with hydration, ships in two languages, and the workspace is managed by Nx.
>
> The part I'd point at first is the permission model. Roles are enforced in three independent places — route guards, a structural directive that keeps buttons out of the DOM entirely, and the API itself. Hiding a button is a courtesy; the server check is the actual boundary."

That answer does three things: it names the product before the technology, it shows you can talk about architecture, and it ends on a specific technical opinion — which invites the follow-up you are best prepared for.

The numbers, if asked

METRIC	VALUE
TypeScript files	98
Total lines (TS + HTML + CSS)	~11,000
Tests	100, across 12 files
Initial bundle	~113 kB gzipped
Screens	Dashboard, Employees, Attendance, Leave, Departments, Admin (5 sub-pages), Settings, Login
Seeded records	16 employees, 7 departments, ~130 attendance rows, 6 leave requests

2. Architecture at a glance

Three layers, one direction

```
features/   the screens          Dashboard, Employees, Leave, Admin …
    |
    v
shared/     reusable UI          Card, Dialog, directives, pipes
    |
    v
core/       no UI at all         models, services, stores, interceptors
```

Imports may only travel downward. `core` may not import a button; `shared` may not import a screen. This is not a convention in a README — it is enforced by `no-restricted-imports` rules in `eslint.config.js`, so `npm run lint` fails on a violation.

> **Say:** "The rule has already caught me once — I put a dependency-injection test in `core` that imported a component from `shared`, and the build failed until I moved it to where it belonged. That's the point of enforcing it rather than documenting it."

Why these three

LAYER	CONTAINS	RULE OF THUMB
`core/`	Data, logic, HTTP, state, auth	Could run without a browser
`shared/`	Presentational components, directives, pipes	Knows nothing about employees
`features/`	Screens wired to routes	One folder per thing a user does

Path aliases

`tsconfig.json` maps `@core/*`, `@shared/*` and `@features/*`, so every import states which layer it crosses. You never see `../../../` and you can tell at a glance whether an import is legal.

3. One request, end to end

If you can tell this story, you can answer most architecture questions. It is what happens when the employee directory loads.

```
1. Router activates  /employees
      |  authGuard checks there is a session
      v
2. EmployeeList component renders, reads EmployeeStore
      |
      v
3. EmployeeStore.load()          <-- NgRx SignalStore
      |  calls
      v
4. EmployeeService.list()        <-- returns an Observable
      |  uses HttpClient
      v
5. INTERCEPTOR CHAIN (outermost first)
      profiling  -> starts a timer
      auth       -> clones the request, adds Bearer token + X-Portal-Role
      cache      -> fresh GET? return it, never touch the network
      error      -> catches failures, raises one toast, re-throws
      mockBackend-> answers from memory (stands in for a server)
      |
      v
6. Response travels back out through the same chain
      |
      v
7. patchState() updates the store's signals
      |
      v
8. Every OnPush view reading those signals re-renders itself
```

The five things this story proves

1. **Routing & guards** — step 1
2. **State management** — steps 3 and 7
3. **RxJS** — step 4
4. **Interceptors** — step 5, all four of them, in a deliberate order
5. **Change detection** — step 8, why OnPush works without any manual calls

Why the interceptor order is what it is

#	INTERCEPTOR	WHY IT SITS THERE
1	profiling	Wraps everything, so it can time a cache hit that never reaches the network
2	auth	Runs before the cache, so cached entries are already role-tagged
3	cache	Serves fresh GETs; any write invalidates it
4	error	Closest to the transport — sees the raw failure before anything transforms it
5	mockBackend	Terminates the chain in place of a server

> **Say:** "The mock backend is an interceptor at the *end* of the chain, not a fake service. That matters: it means auth, caching, error handling and profiling all run for real against it, exactly as they would against a live API. Swapping in a real backend is deleting one file."

4. Every file explained

4.1 Root & bootstrap

FILE	WHAT IT DOES
`src/main.ts`	Browser entry point. Boots the `App` component with `appConfig`.
`src/main.server.ts`	Server entry point for SSR. Same app, server config.
`src/server.ts`	Express server that renders Angular on request and serves static files.
`src/index.html`	The HTML shell. Contains `<base href="/">`, which is what makes clean URLs work on a deep link.
`src/styles.css`	Design tokens (colours, spacing, shadows) plus shared primitives — buttons, status pills, chips. Includes the full light and dark palettes.
`app/app.ts`	The shell component: top bar, navigation, theme toggle, notification bell, account menu. Filters nav items by role.
`app/app.html` / `app.css`	Shell template and styles.
`app/app.config.ts`	**The most important config file.** Registers routing, the interceptor chain, zoneless change detection, hydration, `PathLocationStrategy` and root DI providers.
`app/app.routes.ts`	Root route table. Every screen is lazily loaded.
`app/app.config.server.ts`	Merges server-rendering providers on top of `appConfig`.
`app/app.routes.server.ts`	Per-route render modes — which pages prerender, which render per request, which are client-only.

> **If asked "where would you start reading this project?":** "`app.config.ts`, then `app.routes.ts`. Between them they tell you every cross-cutting decision — how HTTP works, how change detection works, and what screens exist."

4.2 core/ — models, tokens, services

Models — the shape of the data

FILE	WHAT IT DOES
`models/employee.model.ts`	The `Employee` interface and everything about it: roles, statuses, employment types, departments, address. Also small helpers — `fullName()`, `nextEmployeeCode()`, `birthdayWithinDays()`.
`models/hr.model.ts`	Attendance, leave, departments, documents, notifications. Includes `countLeaveDays()`, which counts working days and skips weekends.
`models/announcement.model.ts`	Company announcements. `bodyHtml` is author-written rich text — deliberately untrusted.
`models/api.model.ts`	`Page<T>` envelope, normalised `ApiError`, and `HttpTiming` for the profiler.

Tokens — how dependencies are named

FILE	WHAT IT DOES & WHICH PROVIDER KIND IT SHOWS
`tokens/app-config.token.ts`	`APP_CONFIG` — an `InjectionToken` with a factory. Demonstrates **useValue**-style configuration, typed rather than a magic string.
`tokens/logger.token.ts`	An *abstract class* used as both the token and the contract, plus `LOG_SINK`, a shared array every logger writes into.
`tokens/feature-flags.token.ts`	`FEATURE_FLAGS` with **multi: true** — the root app and the lazy admin module each contribute flags to one array.
`tokens/analytics.token.ts`	Deliberately *never provided*. Exists so `@Optional()` has something real to resolve to `null`.

Services — the logic

FILE	WHAT IT DOES
`services/auth.service.ts`	The session, held in signals. Role hierarchy (`hasRole` compares rank, so ADMIN satisfies a MANAGER check). Handles "remember me" persistence, guarded for SSR and blocked storage.
`services/employee.service.ts`	HTTP for the directory. Where `map` reshapes the API envelope into a plain array.
`services/hr.service.ts`	One thin client for attendance, leave, departments, documents and notifications.
`services/headcount-feed.service.ts`	**Module 6's centrepiece.** A hand-written `new Observable(subscriber => …)` that polls presence, with a teardown function that clears the timer when the last subscriber leaves.
`services/announcement.service.ts`	HTTP for announcements.
`services/notification.service.ts`	Toast messages. Signal-backed, auto-dismissing.
`services/profiling.service.ts`	Collects timings recorded by the profiling interceptor; feeds Admin › System health.
`services/logger.service.ts`	`ConsoleLogger` (root) and `ScopedLogger` (per-scope). The admin module re-provides `Logger` with a scoped instance.
`services/theme.service.ts`	Theme preference: system / light / dark, persisted per browser, SSR-safe.
`services/employee-seed.ts`	All demo data. Employees are written as compact tuples plus a mapper, so adding a field is a one-line change rather than sixteen.

> **On the seed file:** "Recent joining dates are written as `-18`, meaning eighteen days ago, and resolved at load. Hard-coded dates would have made the dashboard's 'new joiners' card permanently empty a few months after I wrote it — which is exactly what happened before I fixed it."

4.3 core/state — the stores

Five NgRx SignalStores. All follow the same four-part shape, which is worth naming out loud:

```
withState    the raw data                employees, loading, error
withComputed derived values, memoised    filtered, activeCount, payrollTotal
withMethods  the only way to change it   load(), create(), update(), remove()
withHooks    lifecycle                   load on first injection
```

STORE	HOLDS
`state/employee.store.ts`	The directory. Filtering, sorting inputs, derived headcount, payroll, recent joiners, upcoming birthdays, distinct designations.
`state/leave.store.ts`	Requests and balances. `decide()` approves or rejects and re-reads balances, because approving spends days.
`state/attendance.store.ts`	Records for a chosen date, plus present/late/absent/on-leave counts and an attendance rate.
`state/department.store.ts`	Department CRUD.
`state/announcement.store.ts`	Announcements, ordered pinned-first then newest.
`state/notification.store.ts`	The bell. Unread count, mark one / mark all read.

Why SignalStore over classic NgRx: "No actions, no reducers, no effects file, no string action types. State is signals, derived data is `computed`, and mutations are ordinary methods. It composes with OnPush for free — a component reads a signal in its template and Angular knows exactly which view to re-check."

Limitation to volunteer: stores mutate through `patchState`, which is synchronous and local. There is no optimistic-update rollback — if a write fails, the store records the error but the UI simply doesn't change. That is fine here; a real app with slow networks would want optimistic updates.

4.4 core/interceptors — the HTTP chain

FILE	WHAT IT DOES
`auth.interceptor.ts`	Clones the request, adds `Authorization: Bearer …` and `X-Portal-Role`. Requests are immutable, hence the clone.
`error.interceptor.ts`	Turns any `HttpErrorResponse` into a normalised `ApiError`, shows one toast, re-throws so callers can still react.
`cache.interceptor.ts`	TTL cache for GETs. A fresh hit short-circuits the chain entirely. Any write clears the cache.
`cache.context.ts`	`CACHE_BYPASS` — an `HttpContextToken` letting one call opt out of the cache.
`profiling.interceptor.ts`	Times every request with `finalize`. A sub-millisecond response is recorded as a cache hit.
`mock-backend.interceptor.ts`	The stand-in server, 436 lines. Split into one handler per resource; enforces role rules on every write.

On the role checks in the mock backend: "The API rejects a delete from a non-admin with a 403, independently of whether the button was shown. There's a test that fires the request anyway and asserts the rejection — because hiding a button is not security."

4.5 core/guards & resolvers

FILE	GUARD TYPE	WHAT IT DOES
`guards/auth.guard.ts`	CanActivate	Requires a session; remembers the target URL so you land where you meant to.
`guards/role.guard.ts`	CanActivate / CanActivateChild	A *factory*: `roleGuard('ADMIN')` returns a guard, so the requirement lives in the route table.
`guards/unsaved-changes.guard.ts`	CanDeactivate	Warns before leaving a dirty form.
`resolvers/employee.resolver.ts`	Resolver	Fetches the record before the detail route activates, so there is no loading state.

Be ready for this one. Both guards return `true` when running on the server. The session lives in `localStorage`, which the server cannot read, so refusing there would bounce every deep link to the login page even for a signed-in user — which is exactly the bug I hit. The browser re-runs the guard after hydration and redirects then. A production system would put the session in an httpOnly cookie, which the server *can* read, and this exception would disappear.

4.6 shared/ — reusable UI

Components

FILE	WHAT IT DOES	WORTH MENTIONING
`card/card.ts`	The panel every screen sits on.	Three `ng-content` slots: title, actions, footer, plus a catch-all. Explicitly `ViewEncapsulation.Emulated`.
`confirm-dialog/confirm-dialog.ts`	Confirmation before any destructive action.	Provides `DialogRef` in **viewProviders** — see the `@Host()` note below.
`confirm-dialog/dialog-ref.ts`	Handle to the surrounding dialog.	Never provided at root; only inside a dialog's own view.
`bar-chart/bar-chart.ts`	Horizontal bar chart for headcount.	Single hue on purpose — see Q&A.
`stat-tile/stat-tile.ts`	The KPI tiles.	Formats currency and thousands separators.
`badge-widget/badge-widget.ts`	Shareable employee badge.	`ViewEncapsulation.ShadowDom` — must resist the host page's styles.
`print-record/print-record.ts`	Printable HR record.	`ViewEncapsulation.None` — `@page` rules cannot be component-scoped.
`notification-bell/notification-bell.ts`	Header bell with unread count.	Uses the click-outside directive to dismiss.
`toast-host/toast-host.ts`	Renders toasts from `NotificationService`.	Lives once, in the shell.

Directives — Module 2 in full

FILE	TYPE	WHAT IT DEMONSTRATES
`has-role.directive.ts`	Structural	`*appHasRole="'ADMIN'; else: x"`. Owns an `<ng-template>` and stamps it only when the role clears the bar. Re-evaluates through an `effect` when the role signal changes.
`feature-flag.directive.ts`	Structural	Written with a classic `@Input` setter, so the `<ng-template>` desugaring is obvious. Reads the `FEATURE_FLAGS` multi-provider.
`tooltip.directive.ts`	Attribute	`Renderer2` builds the bubble node by node; `@HostListener` for show/hide; `@HostBinding` for `tabindex` and `aria-describedby`.
`role-badge.directive.ts`	Attribute	Builds a badge imperatively with `createElement` / `setProperty` / `setStyle` — never `innerHTML`, so there is no injection surface.
`click-outside.directive.ts`	Attribute	`@HostListener('document:click')`. Dismisses menus.
`dialog-close.directive.ts`	Attribute	`@Host()` + `@Optional()`. Finds the dialog it sits inside; inert outside one.

> **The best single thing you can say about DI** — learn this one: "`@Host()` resolves a component's `viewProviders` but *not* its `providers`. Both sit on the same element, but `providers` is one step further out so projected content can reach it too — and that extra step is past where `@Host()` stops. So `ConfirmDialog` declares `DialogRef` in `viewProviders`; if I'd used `providers`, `[appDialogClose]` would silently resolve to `null`. I found that out by testing it, and there are assertions for both directions."

Pipes

`pipes/initials.pipe.ts`	"Aarav Mehta" → "AM". Pure.
`pipes/tenure.pipe.ts`	Joining date → "5.7 yr" or "3 mo". Pure, and takes `now` as an argument so it is testable.

> **Why pipes and not methods:** "A method called from a template re-runs on every change-detection pass. A pure pipe only recomputes when its input changes."

4.7 features/ — the screens

FOLDER	SCREEN	WHAT TO POINT AT
`auth/`	Login	Reactive form, email + password validation, show/hide toggle, remember-me that genuinely persists, demo-account shortcuts.
`dashboard/`	Dashboard	Seven KPI tiles, the department chart, recent joiners, activity feed, birthdays, attendance breakdown, announcements. The "Online now" tile is fed by the custom Observable.
`employees/employee-list/`	Directory	Debounced search, four filters, sortable columns, pagination, role-gated row menus, delete confirmation.
`employees/employee-profile/`	Profile card	**The Module 1 centrepiece.** Both `@Input` / `@Output` and `input()` / `output()`; both `@ViewChild(ren)` and signal queries; local refs; four projection slots.
`employees/employee-detail/`	Employee record	Five tabs — personal, professional, attendance, leave, documents — plus badge and print views.
`employees/employee-form/`	Add / edit	Three sections, ten validation rules, focus jumps to the first invalid field, sticky action bar, unsaved-changes guard.
`employees/employee-documents/`	Documents tab	Upload, download, delete. Metadata round-trips through the API; the file itself never leaves the machine.
`attendance/`	Attendance	Date/status/employee filters. Employees see only their own rows — scoped in the component *and* assumed at the API.
`leave/`	Leave	Balances, an apply form that counts working days live, approve/reject for managers, and approving spends the balance.
`departments/`	Departments	Full CRUD for admins, nested child route for members, delete blocked while people are assigned.
`announcements/`	Announcement panel	Where `DomSanitizer` earns its place — author-written rich text rendered safely.
`admin/`	Admin area	A real lazy `NgModule` with nested routes: overview, announcements, system health, audit log, settings.
`settings/`	Profile & settings	Profile, change password, theme switcher, company info.
`misc/`	403 & 404	Forbidden and not-found pages.

The admin module — why it is the only NgModule left

> **Say:** "Everything in this app is a standalone component except the admin area, which I deliberately kept as an `NgModule` for two reasons. First, the brief asked for a lazy-loaded module. Second, it gives the module its own environment injector — it re-provides `Logger` with an admin-scoped instance, so admin activity is tagged differently in the audit log, and it contributes its own feature flags. You can verify the laziness in the build output: there is a chunk called `admin-module` that is not in the initial bundle."

4.8 Configuration files

FILE	WHAT IT DOES
`nx.json`	Marks `build`, `test`, `lint` cacheable. Analytics disabled.
`project.json`	Replaced `angular.json` when the workspace became Nx. Build, serve, test, lint and i18n targets.
`eslint.config.js`	Angular + accessibility rules, no `any`, no `console.log`, OnPush enforced, and the layer-boundary rules.
`sonar-project.properties`	SonarQube configuration. One exclusion, scoped to specs.
`tsconfig.json`	Strict TypeScript plus the `@core` / `@shared` / `@features` path aliases.
`src/locale/messages.xlf`	31 extracted translatable strings.
`src/locale/messages.fr.xlf`	The French translation.
`scripts/verify-requirements.sh`	`npm run verify` — checks every brief requirement against the source.

5. The nine modules, mapped

Run `npm run verify` in front of them if you want to prove this rather than assert it.

MODULE	REQUIREMENT	WHERE IT LIVES
1 Data binding	Employee Profile component	`employees/employee-profile/`
	@Input / @Output	Same file — decorator *and* signal forms side by side
	ViewEncapsulation	`card.ts` (Emulated), `badge-widget.ts` (ShadowDom), `print-record.ts` (None)
	Local refs, ViewChild(ren)	`employee-profile.ts` — plus `viewChild()` signal queries
	ng-content	Profile (4 slots), Card (4), Dialog (2)
2 Directives	Role-based directive	`has-role.directive.ts`
	Renderer2	`tooltip.directive.ts`, `role-badge.directive.ts`
	HostListener / HostBinding	`tooltip.directive.ts`
	Structural directive	`has-role` and `feature-flag`
3 Change detection	Default vs OnPush	`core/change-detection.spec.ts` — proves the difference by behaviour
	Performance	OnPush everywhere (lint-enforced), zoneless, `@for track`, pure pipes, debounced search, pagination, lazy routes
4 Routing	Lazy Admin NgModule	`features/admin/admin.module.ts`
	Nested routes	`admin-routing.module.ts`, `departments.routes.ts`
	PathLocationStrategy	`app.config.ts` + `<base href>`
5 DI	Hierarchical DI	Admin module re-provides `Logger`; dialog provides `DialogRef`
	All provider kinds	useValue, useClass, useFactory, useExisting, multi — all present
	@Optional, @Host	`dialog-close.directive.ts`, `analytics.token.ts`
6 RxJS	Custom observable	`headcount-feed.service.ts` — feeds the dashboard's "Online now" tile
	Observer object	`dashboard.ts` — explicit next/error/complete
	map, filter, takeUntil	Services and `employee-list.ts` (+ debounceTime, distinctUntilChanged)
7 Security	DomSanitizer	`announcement-panel.ts`, `admin-announcements.ts`
	XSS prevention	Seed data carries a real payload; a test asserts it never reaches the DOM
8 Interceptors	Auth, error, cache, profiling	`core/interceptors/` — all four, order justified
9 Modern Angular	Standalone	Every component
	Signals	signal, computed, effect, linkedSignal throughout
	i18n	31 messages, French locale, build-time translation
	SSR	`server.ts` + per-route render modes
	State management	Six SignalStores
	Nx	`nx.json`, `project.json`, cached targets

Sonar	`sonar-project.properties` + ESLint doing it locally

6. Questions you will be asked

Warm-up

Why standalone components instead of NgModules?

Less ceremony and honest dependencies — a component lists exactly what its template uses, so you can see at a glance what it needs, and tree-shaking gets easier. NgModules made you register things far away from where they were used. The one module I kept is the admin area, on purpose, to demonstrate lazy module loading and to get its own environment injector.

What is zoneless change detection?

Traditionally Angular used Zone.js, which monkey-patches browser APIs — `setTimeout`, event listeners, XHR — so it can tell when something *might* have changed and check everything. Zoneless removes that. Change detection is instead driven by signals being set, template event handlers firing, and async pipe emissions. It's less magic, less overhead, and it only works well if your state is in signals — which it is here.

Signal vs computed vs effect?

`signal` is a value that notifies readers when it changes. `computed` derives a value from other signals, is memoised, and only recalculates when something it read actually changed. `effect` runs a side effect when its dependencies change — for talking to the outside world, like writing the theme to `localStorage`. The rule is: never use an effect to compute a value another template reads; that's what `computed` is for.

The ones that separate candidates

Explain OnPush, and the trap it introduces.

With Default, Angular checks a view on every change-detection pass whether or not anything it shows changed. With OnPush it checks only when: an `@Input` gets a *new reference*, an event fires from inside the view, a signal read in the template changes, or `markForCheck()` is called.

The trap is the first one. If you mutate an object in place — `employee.salary += 50` — the reference never changed, so an OnPush view keeps showing the old number while a Default view updates. That's precisely why `EmployeeStore.update()` maps to a new object instead of mutating. I have a test that mounts one of each strategy and asserts Default reads 150 while OnPush still reads 100.

Why is the mock backend an interceptor and not a fake service?

Because a fake service would bypass the very thing I'm trying to demonstrate. Putting it at the end of the interceptor chain means auth, caching, error handling and profiling all execute for real against it — the same code paths that would run against a live API. It also means swapping in a real backend is deleting one file, with nothing else to change.

Walk me through your interceptor order.

Outermost first: profiling, auth, cache, error, mock backend. Profiling wraps everything so it can time a cache hit that never reaches the network. Auth runs before the cache so cached entries are already role-tagged. Error sits closest to the transport so it sees the raw failure before anything transforms it. Each position is a decision, not an accident.

How is role-based access enforced?

Three independent layers. Route guards block navigation. The `*appHasRole` structural directive removes buttons from the DOM entirely — not disabled, absent; there's a test asserting no disabled buttons exist. And the API rejects the request regardless: try to delete as a manager and you get a 403 with "only administrators may delete employee records."

The first two are user experience. Only the third is security.

Where do you use @Host, and why does it matter?

On `[appDialogClose]`. That directive is only meaningful inside a dialog, so it looks for a `DialogRef` and stops at the dialog's view boundary rather than walking to the root — it can't accidentally close some ancestor dialog. Combined with `@Optional()` it's inert outside a dialog instead of throwing. It's the same contract `formControlName` has with its parent form.

The subtlety I'd add: `@Host()` sees `viewProviders` but not `providers`. I learned that empirically — my first version used `providers` and resolved to `null` every time.

Why not just sanitize with bypassSecurityTrustHtml?

Backwards — `bypassSecurityTrust*` *disables* sanitisation. This app doesn't call it anywhere. Announcement bodies are rich text written by managers, so they're untrusted; binding them with `[innerHTML]` runs Angular's sanitiser, which strips `<script>`, `onerror` and `javascript:` URLs while keeping legitimate formatting. The seed data contains a real payload so this is demonstrable, and a test asserts it never reaches the DOM — including in server-rendered HTML.

Why is your chart one colour?

Because it shows one measure across categories that the axis already names. A second colour would encode nothing, and multi-hue palettes have to clear colourblind-safety thresholds — so I'd be taking on a real accessibility cost for zero information gain. I validated the palette against this app's own light and dark surfaces. The attendance breakdown is stat tiles rather than a four-colour chart for the same reason: those status hues fail the separation floor when used as a series, but they're fine as labelled tiles where the label carries the meaning.

What does Nx actually give you here?

Honestly, not much at this size — and I'd say that rather than oversell it. What it does give me is task caching: a second `npm run build` with no changes goes from about four seconds to half a second, because Nx knows nothing changed. It also brings the layered-architecture idea, which I enforce with lint rules. Nx earns its keep with fifteen apps in a monorepo; with one app it's mostly the caching.

Your test coverage is 75%. Why not higher?

The 25% is almost entirely feature components — screens. What is covered is the part where bugs are expensive: the whole interceptor chain, every store, the guards, the directives, DI resolution, and the profile component. I'd rather have 100 meaningful tests than 400 that assert a template rendered. If I had another day I'd add component tests for the attendance and leave screens, because those have real logic in them.

If they push on weaknesses

There's no real backend. Isn't that a cop-out?

It's a deliberate trade. The alternative was spending time on a server that demonstrates nothing about Angular. What I did instead was make the fake indistinguishable from the real thing at the boundary — it goes through HTTP, through every interceptor, returns real status codes, and enforces authorization. The data resets on reload, which I'd call out to any user.

Authentication has no password check.

Correct, and I wouldn't claim otherwise. There's no credential store and no token verification — sign-in matches an email against three demo accounts. What *is* real is everything after authentication: the role hierarchy, the three enforcement layers, session persistence, and the guard behaviour. Adding real auth would mean an identity provider and httpOnly cookies, which would also let me delete the SSR workaround in the guards.

Show me something that went wrong.

Several things, and I'd pick the row action menu. Each of the ten table rows registered its own "did you click outside me?" listener. Clicking row one's menu looked *outside* to the other nine, whose handlers closed it on the same click — so the menu never opened at all, and I hadn't noticed because I'd only screenshotted it, not clicked it. I proved the cause by dispatching a non-bubbling click, which opened it fine. One listener now wraps the table, with four regression tests.

7. Live demo script

Eight minutes, in this order. Every step shows something a requirement asked for.

```
npm start            →  http://localhost:4200
```

#	DO THIS	SAY THIS
1	Land on `/login`. Click **Show** on the password. Sign in as **ADMIN**.	"Reactive form, email and length validation, show/hide toggle. Remember me persists the session, so a refresh doesn't sign you out."
2	Dashboard. Point at the tiles and the chart. Wait ~5s and note **Online now** changing.	"Seven KPIs, all derived with `computed`. The chart is single-hue by design. The Online now tile is fed by a hand-written Observable that stops polling when you leave the page."
3	Employees. Type in search — pause on the delay. Sort a column. Page forward.	"Search is debounced 250ms through an RxJS Subject, so it doesn't re-filter on every keystroke."
4	Open a row's `⋯` menu → **Remove**. Cancel the dialog.	"Destructive actions always confirm. The dialog provides a DialogRef in viewProviders, which is how the 'Not now' button finds it with `@Host()`."
5	Click an employee. Walk the five tabs.	"Personal, professional, attendance, leave, documents. The profile card is the component where I demonstrate both generations of Angular's input, output and query APIs."
6	**Add employee**. Press Create with the form empty.	"Ten validation rules, inline messages, and focus jumps to the first invalid field rather than leaving you to hunt."
7	Leave → **Approve** a pending request. Watch the balance tile drop.	"Approving spends the allowance — the store re-reads balances after the decision."
8	Admin → **System health**. Press GET twice.	"The second request comes back as a CACHE row in under a millisecond — it never reached the network. These numbers are collected by the profiling interceptor."
9	**The finale.** Admin → Settings → switch role to **EMPLOYEE**. Go back to Employees.	"Watch what disappears: the Admin tab, the salary column, Add employee, the whole row menu. Nothing is greyed out — it's removed from the DOM. And the API would refuse anyway."
10	Toggle the theme icon in the header.	"System, light and dark. The dark palette is selected for the dark surface, not an automatic inversion."

If you have a terminal on screen

```
npm run verify     # 72 module requirements + 73 feature items, all passing
npm test           # 100 tests
npm run build      # then run it again — Nx cache, 4.2s → 0.6s
```

8. Known limitations

Say these before you are asked. Each one is a trade you made on purpose, not something you missed.

LIMITATION	HOW TO FRAME IT
No real backend — data resets on reload	Deliberate. The mock lives at the end of the interceptor chain so everything above it is real. Swapping in a live API is deleting one file.
Authentication is simulated — no password check	The parts that demonstrate Angular — guards, role hierarchy, three-layer enforcement, session persistence — are all real. Only credential verification is missing.
Guards allow through during SSR	The session lives in localStorage, which the server can't read. An httpOnly cookie would fix it properly and let me delete the exception.
75% test coverage	Core logic is well covered; screens are not. I'd add attendance and leave component tests next.
Sonar can't actually run here	The config is checked in and correct, but the scanner needs a SonarQube server or a SonarCloud token. ESLint does the equivalent checks locally on every commit.
Nx is over-specified for one app	I'd say this before they do. It gives real caching, but its value is in monorepos. It's here because the brief asked for it.
Document upload is metadata only	No file storage backend. The record round-trips through the API; the bytes stay on your machine.
Only two locales	English and French, translated at build time. Adding a third is one XLF file and one line of config.

> **A good closing line:** "The thing I'd change first, given more time, is replacing the simulated auth with a real identity provider using httpOnly cookies — because that single change would also let me delete the SSR workaround in the guards, which is the one piece of the codebase I'm least happy with."

9. One-page cheat sheet

Commands

```
npm start          dev server              npm test        100 tests
npm run build      production + SSR        npm run lint    ESLint + a11y
npm run verify     requirement checklist   npm run build:i18n   en-US + fr
npm run serve:ssr  run the built server    npx nx graph    task graph
```

Not `ng build` — this is an Nx workspace; `angular.json` is gone.

Demo accounts **password: portal123**

EMAIL	ROLE	CAN DO
`employee@acme.io`	EMPLOYEE	Own profile, own attendance, apply for leave, browse
`manager@acme.io`	MANAGER	+ salaries, all attendance, approve leave, add/edit, admin area
`admin@acme.io`	ADMIN	+ delete, department CRUD, settings

The five sentences to memorise

1. **Architecture:** "features → shared → core, one direction, enforced by lint."
2. **State:** "SignalStore — withState, withComputed, withMethods, withHooks. No actions, no reducers."
3. **HTTP:** "Profiling wraps everything, auth before cache, error nearest the transport, mock backend terminates."
4. **Security:** "Three layers — guards, the directive removes buttons from the DOM, and the API refuses anyway. Only the third is security."
5. **OnPush:** "Checked only on a new input reference, an event from inside, a signal read, or markForCheck. Mutating in place is the trap."

Numbers

98 TypeScript files	~11,000 lines	100 tests, 12 files
113 kB gzipped initial	16 employees, 7 departments	31 translated messages
6 SignalStores	6 custom directives	5 interceptors

If you blank

Open `src/app/app.config.ts` and talk through it line by line. It contains the routing setup, the whole interceptor chain, zoneless change detection, hydration, `PathLocationStrategy` and the DI providers — five of the nine modules are visible in one 73-line file.

Employee Management Portal · Viva Guide · Regenerate with `node scripts/build-viva-pdf.mjs`